

CROP DISEASE DETECTION USING CNN

(A Research-Based Project Report)

Name: Praveena Chinnusamy

College: Gnanamani College

Internship: Data Reveal

Date: 18 March 2026

Paper published on March -April 2025

Abstract:

This research project focuses on developing an intelligent system for

detecting crop diseases using image processing and deep learning

techniques. The system uses a Convolutional Neural Network (CNN) to

analyze leaf images and identify diseases at an early stage. The

integration of a Flutter-based frontend and Flask-based backend enables

real-time prediction and user interaction. This approach helps in

reducing crop loss and improving agricultural productivity.

1. Introduction

Agriculture plays a vital role in the economy. One of the major

challenges faced by farmers is crop diseases. Manual detection is time consuming and requires expertise. This project provides an automated

solution using Artificial Intelligence to detect diseases from leaf images.

2. Problem Statement

Farmers often fail to detect diseases at an early stage, leading to

significant crop loss. There is a need for an efficient and user-friendly

system that can identify diseases quickly and accurately.

3. Objective

The objective of this project is to develop a smart system that detects

crop diseases using CNN, reduces crop loss, and assists farmers in

taking timely action.

4. Methodology

The system follows a structured approach:

1. Image Collection: Leaf images are collected.

2. Preprocessing: Images are resized and normalized.
3. Model Training: CNN model is trained on dataset.
4. Prediction: Model predicts disease.
5. Deployment: Integrated with Flask API and Flutter app.

5. System Architecture

Frontend (Flutter) → Backend (Flask API) → CNN Model
→ Result

The frontend captures user input, the backend processes data, and the AI

model generates predictions.

6. Technologies Used

Frontend

Flutter:

Used to build the mobile application interface

Allows users to upload images and view results

Backend

Flask:

Used to create API (server)

Handles image processing and communication

Artificial Intelligence

Convolutional Neural Network:

Used to detect crop diseases from images

Learns patterns like spots, color changes

Programming Language

Python:

Used for backend development and AI model

Dart:

Used for Flutter app development

Image Processing

OpenCV:

Used for image handling and processing

Pillow:

Used to open and resize images

NumPy:

Used to convert image into array format

Deep Learning Framework

TensorFlow:

Used to build and train CNN model

Keras:

High-level API for building neural networks

API Communication

REST API:

Used to connect frontend and backend

Transfers data (image & result)

7. Working Process

The working process of the Crop Disease Detection System involves

multiple steps, starting from user interaction to final prediction using the

CNN model.

Step 1: Image Upload through Flutter App

The process begins when the user (farmer) opens the mobile application

developed using Flutter.

The app provides an option to:

- Capture a live image using the camera
- Or upload an existing image from the gallery
- The selected image is displayed on the screen for confirmation

before sending.

Purpose:

- To collect real-time leaf images directly from the user in an easy and
- user-friendly way.

Step 2: Image Sent to Flask API

Once the image is selected, it is sent to the backend server using a REST

API.

- The Flutter app sends the image using an HTTP POST request
- The image is usually sent in:
 - Multipart/form-data format
 - Along with necessary headers
- The Flask server receives the image file and stores it temporarily.

Purpose:

To transfer the image from frontend (mobile app) to backend (server) for processing.

Step 3: Image Preprocessing and Conversion

After receiving the image, the backend processes it before giving it to the CNN model.

Preprocessing steps include:

Resizing:

Image is resized (e.g., 224×224 pixels) to match model input size

Normalization:

Pixel values are scaled (0–255 → 0–1) for better model

performance

Conversion to Array:

Using NumPy, the image is converted into a numerical array

format

Dimension Expansion:

Image is reshaped to include batch size (e.g., $1 \times 224 \times 224 \times 3$)

Purpose:

To make the image compatible with the CNN model input format.

Step 4: CNN Model Prediction

The processed image is passed into the trained Convolutional Neural

Network.

What happens inside CNN:

- The model analyzes:
- Leaf color variations
- Spots and patterns
- Texture differences
- CNN automatically extracts features using:
- Convolution layers
- Pooling layers

- Final output layer gives probabilities for each disease class

Example Output:

- Healthy: 10%
- Early Blight: 85%
- Late Blight: 5%

Predicted Result:

Early Blight (Highest Probability)

Purpose:

To classify the image into a specific disease category using learned patterns.

Step 5: Result Sent Back to Flutter App

After prediction, the backend sends the result back to the frontend.

- Flask API returns:
- Disease name
- Confidence score (%)

Purpose:

To communicate prediction results back to the user.

Step 6: Result Display to User

Finally, the Flutter app displays the result in a user-friendly format.

- Shows:
- Disease name
- Confidence level
- (Optional) Suggestions or remedies
- UI may include:
- Text output
- Color indicators (Healthy/Diseased)

Purpose:

To provide easy understanding and actionable insights for the farmer.

8. Advantages

1. Early Disease Detection

This system helps farmers identify plant diseases at an early stage using image analysis.

Normally, farmers may not notice small symptoms, but the CNN model can detect even minor changes in leaves.

Benefit:

Early detection prevents the disease from spreading to other plants.

2. Reduces Crop Loss

When diseases are detected early, farmers can take quick action like applying pesticides or removing infected plants.

Benefit:

This reduces damage and increases overall crop yield.

3. User-Friendly Interface

The mobile application (built using Flutter) is designed in a simple way:

- Upload image
- Click button
- Get result

Benefit:

Even farmers with basic smartphone knowledge can use it easily.

4. Fast and Accurate Predictions

The CNN (Convolutional Neural Network) processes the image quickly and gives results within seconds.

Benefit:

- Saves time
- Provides reliable predictions compared to manual checking

9. Future Scope

1. Improve Accuracy

Currently, the model is trained with a limited dataset.
In the future:

- Add more plant images
- Include different disease variations

Result:

Higher accuracy and better prediction performance.

2. Weather Prediction Integration

Diseases often depend on weather conditions like:

- Temperature
- Humidity
- Rainfall

By adding weather data:

Benefit:

System can **predict diseases before they appear** (preventive system).

3. Location-Based Detection (GPS)

Using GPS, the system can:

- Identify the farmer's location
- Analyze region-specific diseases

Benefit:

Provides **more accurate and localized solutions**.

4. Multi-Language Support

The app can be developed in multiple languages like:

- Tamil
- Hindi
- English

Benefit:

Farmers from different regions can use it easily without language barriers.

5. Offline Usage

In rural areas, internet access may be limited.

Future improvement:

- Allow app to work without internet

Benefit:

Farmers can use the system anytime, anywhere.

6. Disease Severity Level

Instead of just showing the disease name, the system can classify severity:

- Low
- Medium
- High

Benefit:

Helps farmers understand **how serious the disease is** and take proper action.

7. Suggest Solutions

The system can recommend:

- Suitable pesticides
- Fertilizers
- Treatment methods

Benefit:

Provides **complete guidance**, not just detection.

Overall System Flow:

User (Mobile App - Flutter)

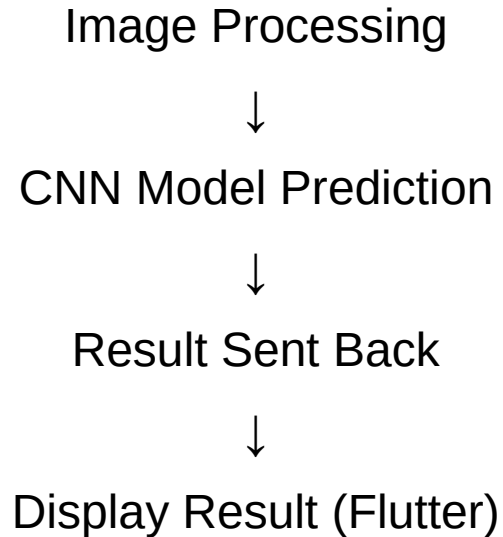


Upload Leaf Image



API Request (Flask Backend)





Overall System Flow

Step-by-Step Working:

1. **User (Mobile App – Flutter)**

The farmer opens the mobile app.

2. **Upload Leaf Image**

The user captures or uploads an image of the crop leaf.

3. **API Request (Flask Backend)**

The app sends the image to the backend server using an API.

4. **Image Processing**

The system:

- Resizes the image
- Converts it into numerical format (array)

5. **CNN Model Prediction**

The trained CNN model analyzes the image and:

- Identifies patterns
- Classifies disease

6. **Result Sent Back**

The prediction result is sent back to the mobile app.

7. **Display Result (Flutter App)**

The app shows:

- Disease name
- Confidence level
- (Future: solution suggestions)

9. **Conclusion**

This project shows how **Artificial Intelligence (AI)** can be effectively used in agriculture.

The system:

- Detects crop diseases automatically
- Reduces dependency on manual inspection
- Saves time and effort for farmers

Using **CNN (Deep Learning)**, the model can analyze plant images with high accuracy and provide quick results.

Final Impact:

- Improves crop quality
- Increases agricultural productivity
- Supports farmers with smart technology

